

← Troubleshoot scaling i... ... ⌚ ⚙️ 📝

I am currently doing an Introduction to Artificial Intelligence assignment.

I don't want you to make the fixes or any implementation, just help me to understand implementation, debug, and point me to potential issues and guide me to understand so that I can fix them on my own.

Now I have this issue regarding my scaling. I am trying to fit and transform my training set (that was split), but there is a large value preventing it from ...

Show more ▾

Worked for 31s >

The issue is probably not "a large value" in the normal outlier sense. Your traceback says:

```
Input X contains infinity or a  
value too large for  
dtype('float64')
```

In your notebook, the likely source is:

python



```
taxi_cleaned['tip_rate_proxy']
```



Now I have this issue regarding my scaling. I am trying to fit and transform my training set (that was split), but there is a large value preventing it from scaling

← Troubleshoot scaling i...

python

```
fare_amount / 0
```

becomes `inf`, and `RobustScaler` refuses to fit.

To debug it yourself, check before scaling:

python

```
import numpy as np
```

```
X_train[numeric_columns].isin([
```

Then locate the bad rows:

python

```
bad_rows = X_train[np.isinf(X_t  
bad_rows[['fare_amount', 'trip_
```

Also check extreme values:

python

```
X_train[numeric_columns].descri
```

The conceptual fix is: handle impossible/invalid taxi rows **before** scaling. For this dataset, `trip_distance` `<= 0` and probably `fare_amount` `<= 0` are invalid for a fare-per-mile feature. Your assignment even hints at that in the TODO.



One more issue I noticed later in the

← Troubleshoot scaling i...

The conceptual fix is: handle impossible/invalid taxi rows **before** scaling. For this dataset, `trip_distance` `<= 0` and probably `fare_amount` `<= 0` are invalid for a fare-per-mile feature. Your assignment even hints at that in the TODO.

One more issue I noticed later in the same cell:

python

```
X_test_scaled[numeric_columns]
```

That should conceptually be assigning into your copied dataframe, not into the NumPy scaled array. So after you fix the infinity problem, that line will likely cause another issue.

Your scaling logic is otherwise right:

python

```
scaler.fit_transform(X_train[nu  
scaler.transform(X_val[numeric_  
scaler.transform(X_test[numeric
```

Fit only on training data; transform validation/test using the training-set scaler.

It means I have to fix the invalid rows check in cell 9?



Yes. The invalid-row check should

← Troubleshoot scaling i... ... ↺ ⚙️ ✎

It means I have to fix the invalid rows check in cell 9?

Yes. The invalid-row check should happen before you create `tip_rate_proxy`, so cell 9 is the right place if that is your cleaning/preprocessing cell.

Right now you check for missing values, but not invalid zero/negative values:

```
python
taxi['trip_distance'].notna()
taxi['fare_amount'].notna()
taxi['tip_amount'] >= 0
```

You should reason about adding conditions like:

```
python
taxi['trip_distance'] > 0
taxi['fare_amount'] > 0
```

because your engineered feature divides by `trip_distance`:

```
python
fare_amount / trip_distance
```

So the key idea is:

1. Remove or han  rows where `trip_distance <= 0`.

← Troubleshoot scaling i... ... ↺ ⚙️ ✎

```
python
taxi['trip_distance'].notna()
taxi['fare_amount'].notna()
taxi['tip_amount'] >= 0
```

You should reason about adding conditions like:

```
python
taxi['trip_distance'] > 0
taxi['fare_amount'] > 0
```

because your engineered feature divides by `trip_distance`:

```
python
fare_amount / trip_distance
```

So the key idea is:

1. Remove or handle rows where `trip_distance <= 0`.
2. Then create `tip_rate_proxy`.
3. Then split.
4. Then fit the scaler on `X_train` only.

A useful check after cell 9 is:

```
python
xi_cleaned['trip_distance'] <= 0]
```

That should be `0` before you calculate `tip_rate_proxy`.



← Troubleshoot scaling i... ... ⌚ ⚙️ 📄

Now I need to train a regressor

For this assignment, start with a simple baseline regressor first. A good first choice is `LinearRegression`, because it lets you check whether your preprocessing pipeline works before trying more flexible models.

Assuming your prepared datasets are:

```
python
X_train_ready
X_val_ready
X_test_ready
y_train
y_val
y_test
```

you can train like this:

```
python
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train_ready, y_train)
```

Then predict:

```
python
y_train_pred = model.predict(X_train_ready)
y_val_pred = model.predict(X_val_ready)
y_test_pred = model.predict(X_test_ready)
```

← Troubleshoot scaling i... ... ⌚ ⚙️ 📄

```
python
from sklearn.metrics import mean_squared_error, r2_score

train_rmse = mean_squared_error(y_train, y_train_pred)
val_rmse = mean_squared_error(y_val, y_val_pred)
test_rmse = mean_squared_error(y_test, y_test_pred)

train_r2 = r2_score(y_train, y_train_pred)
val_r2 = r2_score(y_val, y_val_pred)
test_r2 = r2_score(y_test, y_test_pred)

print("Train RMSE:", train_rmse)
print("Validation RMSE:", val_rmse)
print("Test RMSE:", test_rmse)

print("Train R²:", train_r2)
print("Validation R²:", val_r2)
print("Test R²:", test_r2)
```

One important thing: if your categorical columns are still strings, `LinearRegression` will fail. Models need all numeric input. So before fitting, check:

```
python
X_train_ready.dtypes
```

If you still see `object` columns like `store_and_fwd_flag`, you need to encode categoricals first, for example with `pd.get_dummies`, before training.

Also, since your target is `tip_amount`, make sure `tip_amount` is not inside `X_train_ready`. You already dropped it

← Troubleshoot scaling i... ⋮ ⌚ ⚙️ 📄



Is this normal?

Yes, that is normal. It is a **warning**, not an error.

This warning comes from Seaborn. It means that in newer versions, using:

```
python
palette="viridis"
```

without also setting `hue` will no longer be supported.

Your plot still works now. To silence the warning and make it future-compatible, change:

```
python
sns.countplot(data=obesity, y="
```

to:

```
python
sns.countplot(
    data=obesity,
    y="NObeyesdad",
    hue="NObeyesdad",
    palette="viridis",
    legend=False
```

← Troubleshoot scaling i... ⋮ ⌚ ⚙️ 📄

```
sns.countplot(data=obesity, y="
```

to:

```
python
sns.countplot(
    data=obesity,
    y="NObeyesdad",
    hue="NObeyesdad",
    palette="viridis",
    legend=False
)
```

That tells Seaborn: "color each `NObeyesdad` category separately, but don't show a redundant legend."

Also, this line:

```
python
obesity.shape, obesity.head, ob
```

does not actually call `head`, `info`, or `describe`. You are referencing the methods, not running them. Use parentheses:

```
python
obesity.shape
obesity.head()
obesity.info()
obesity.describe()
obesity.isna().sum()
```

The class distribution itself looks

Yes. For the obesity classification section, you need to encode categorical columns because scikit-learn models need numeric input.

First identify the target and feature columns:

```
python

X = obesity.drop(columns=["NObeyesdad"])
y = obesity["NObeyesdad"]
```

Then separate categorical and numeric columns:

```
python

categorical_columns = X.select_dtypes(include=["object"]).columns
numeric_columns = X.select_dtypes(exclude=["object"]).columns
```

For a simple notebook-friendly approach, use `pd.get_dummies` :

```
python

X_encoded = pd.get_dummies(
    X,
    columns=categorical_columns,
    drop_first=True
)
```

Then check that everything is numeric:

```
python

X_encoded.dtypes
```

You should no longer see `object` .

After that, split:

```
python

from sklearn.model_selection import train_test_split
```

Then check that everything is numeric:

```
python  
  
X_encoded.dtypes
```

You should no longer see `object` .

After that, split:

```
python  
  
from sklearn.model_selection import train_test_split  
  
X_train_val, X_test, y_train_val, y_test = train_test_split(  
    X_encoded,  
    y,  
    test_size=0.20,  
    random_state=RANDOM_STATE,  
    stratify=y  
)  
  
X_train, X_val, y_train, y_val = train_test_split(  
    X_train_val,  
    y_train_val,  
    test_size=0.25,  
    random_state=RANDOM_STATE,  
    stratify=y_train_val  
)
```

Use `stratify=y` here because this is classification and you want the class proportions to stay similar across train/validation/test.

Then scale only the numeric columns, not the dummy columns. Since `numeric_columns` came from the original `X` , those names still exist inside `X_encoded` .



For binary columns and ordinal-ish, how do I approach it?

For this obesity dataset, treat them differently:

Binary Columns

Use simple 0/1 mapping.

Examples likely include:

```
python

Gender
family_history_with_overweight
FAVC
SMOKE
SCC
```

For yes/no columns:

```
python

obesity["family_history_with_overweight"] = obesity["family_
    "yes": 1,
    "no": 0
})
```

Same idea for FAVC , SMOKE , SCC .

For Gender , you can map it too:

```
python

obesity["Gender"] = obesity["Gender"].map({
    "Female": 0,
    "Male": 1
})
```

They often contain values like:

```
python

"no", "Sometimes", "Frequently", "Always"
```

You can map them in order:

```
python

ordinal_map = {
    "no": 0,
    "Sometimes": 1,
    "Frequently": 2,
    "Always": 3
}

obesity["CAEC"] = obesity["CAEC"].map(ordinal_map)
obesity["CALC"] = obesity["CALC"].map(ordinal_map)
```

Nominal Columns

These have no natural order, so use one-hot encoding.

Likely nominal:

```
python

MTRANS
```

Use:

```
python

obesity_encoded = pd.get_dummies(
    obesity,
    columns=["MTRANS"],
    drop_first=True
)
```

Target Column

Target Column

Do not one-hot encode `NObeyesdad` for most scikit-learn classifiers. Keep it as labels:

```
python

y = obesity["NObeyesdad"]
```

or use `LabelEncoder` if the model requires numeric labels, but many classifiers accept string labels directly.

A clean approach:

```
python

obesity_clean = obesity.copy()

binary_map = {"yes": 1, "no": 0}

for col in ["family_history_with_overweight", "FAVC", "SMOKE"]:
    obesity_clean[col] = obesity_clean[col].map(binary_map)

obesity_clean["Gender"] = obesity_clean["Gender"].map({
    "Female": 0,
    "Male": 1
})

ordinal_map = {
    "no": 0,
    "Sometimes": 1,
    "Frequently": 2,
    "Always": 3
}

for col in ["CAEC", "CALC"]:
    obesity_clean[col] = obesity_clean[col].map(ordinal_map)
```

Then one-hot encode `CAEC` & `CALC` if you want to flatten the target

Wait, does it mean I have to drop NObeyesdad since its a potential leak?

Yes, you must drop NObeyesdad from the features.

But don't delete it entirely. It is your target.

For supervised classification:

```
python
X = obesity_encoded.drop(columns=["NObeyesdad", "NObeyesdad_"])
y = obesity_encoded["NObeyesdad"]
```

or, if you keep the target as text:

```
python
X = obesity_encoded.drop(columns=["NObeyesdad"])
y = obesity_encoded["NObeyesdad"]
```

The leak would be using NObeyesdad inside X .

So the logic is:

```
text
NObeyesdad = answer you are trying to predict
X = input features
y = correct answer/label
```

If NObeyesdad stays in X , the model can just learn the answer directly, which is target leakage.

BMI is different. BMI is calculated from Height and Weight , which are feature columns. It is very strongly related to the target, but it is not the actual target label itself. You can include it if ↓ explain that it may make the classification easier because obesity levels are closely tied to BMI.

So do I do this before the domain feature bit?

You can do it either before or after creating the domain feature, but the cleanest order is:

1. Start from your encoded dataset.
2. Create the domain feature, for example `BMI`.
3. Encode the target.
4. Separate `X` and `y`.
5. Drop the target columns from `X`.

So yes, create BMI **before** separating `X` and `y`, because BMI is a feature.

Example flow:

```
python
obesity_encoded["BMI"] = obesity_encoded["Weight"] / (obesity_
```

Then encode the target:

```
python
from sklearn.preprocessing import LabelEncoder

label_encoder = LabelEncoder()
obesity_encoded["NObeyesdad_encoded"] = label_encoder.fit_tr
    obesity_encoded["NObeyesdad"]
)
```

Then separate:

```
python
X = obesity_encoded.drop(columns=["NObeyesdad", "NObeyesdad_
```

So yes, create BMI before separating `X` and `y`, because BMI is a feature.

Example flow:

```
python
obesity_encoded["BMI"] = obesity_encoded["Weight"] / (obesity_
```

Then encode the target:

```
python
from sklearn.preprocessing import LabelEncoder

label_encoder = LabelEncoder()
obesity_encoded["NObeyesdad_encoded"] = label_encoder.fit_tr
    obesity_encoded["NObeyesdad"]
)
```

Then separate:

```
python
X = obesity_encoded.drop(columns=["NObeyesdad", "NObeyesdad_
y = obesity_encoded["NObeyesdad_encoded"]
```

Why this order makes sense:

- BMI uses Weight and Height, which are input features.
- NObeyesdad is the answer label.
- NObeyesdad_encoded is just the numeric version of the answer label.
- Neither NObeyesdad nor NObeyesdad_encoded should be inside X.

So don't drop NObeyesdad before label encoding unless you already saved it separately as `y`.



Alright, now on classification training

For classification training, start with a simple baseline model first. Since your features are scaled and numeric, a good first choice is `LogisticRegression` .

Use your scaled datasets:

```
python

X_train_scaled
X_val_scaled
X_test_scaled
y_train
y_val
y_test
```

Train:

```
python

from sklearn.linear_model import LogisticRegression

clf = LogisticRegression(
    max_iter=1000,
    random_state=RANDOM_STATE
)

clf.fit(X_train_scaled, y_train)
```

Predict:

```
python

y_train_pred = clf.predict(X_train_scaled)
y_val_pred = clf.predict(X_val_scaled)
y_test_pred = clf.predict(X_test_scaled)
```

F

python

```
from sklearn.linear_model import LogisticRegression

clf = LogisticRegression(
    max_iter=1000,
    random_state=RANDOM_STATE
)

clf.fit(X_train_scaled, y_train)
```

Predict:

python

```
y_train_pred = clf.predict(X_train_scaled)
y_val_pred = clf.predict(X_val_scaled)
y_test_pred = clf.predict(X_test_scaled)
```

Evaluate:

python

```
from sklearn.metrics import accuracy_score, classification_r

print("Train accuracy:", accuracy_score(y_train, y_train_pre
print("Validation accuracy:", accuracy_score(y_val, y_val_pr
print("Test accuracy:", accuracy_score(y_test, y_test_pred))

print("Validation classification report:")
print(classification_report(y_val, y_val_pred))
```

For the confusion matrix:

python

```
import seaborn as sns
import matplotlib.pyplot as plt
```



```
plt.show()
```

If you want class names instead of numbers:

```
python

class_names = label_encoder.classes_

cm = confusion_matrix(y_test, y_test_pred)

plt.figure(figsize=(9, 7))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Logistic Regression")
plt.show()
```

After that, you can compare with a stronger model like

`RandomForestClassifier`. But train `LogisticRegression` first as your baseline.

What about the accuracy and macro-f1? That's before the confusion matrix

Yes, compute **accuracy** and **macro-F1** before the confusion matrix.

Accuracy tells you overall percent correct. Macro-F1 is better for multi-class classification because it gives equal weight to each class.

After predictions:

```
y_train_pred = clf.predict(X_train_scaled)
y_val_pred = clf.predict(X_val_scaled)
y_test_pred = clf.predict(X_test_scaled)
```

Compute metrics:

```
python

from sklearn.metrics import accuracy_score, f1_score

train_accuracy = accuracy_score(y_train, y_train_pred)
val_accuracy = accuracy_score(y_val, y_val_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

train_macro_f1 = f1_score(y_train, y_train_pred, average="macro")
val_macro_f1 = f1_score(y_val, y_val_pred, average="macro")
test_macro_f1 = f1_score(y_test, y_test_pred, average="macro")

print("Train accuracy:", train_accuracy)
print("Validation accuracy:", val_accuracy)
print("Test accuracy:", test_accuracy)

print("Train macro-F1:", train_macro_f1)
print("Validation macro-F1:", val_macro_f1)
print("Test macro-F1:", test_macro_f1)
```

Then after that, do:

```
python

from sklearn.metrics import classification_report, confusion_matrix
```

The `classification_report` also includes macro-F1, but printing it separately is useful because your TODO specifically asks for accuracy and macro-F1.

📄 👍 💬 ↗️ 1:54 AM



I'm using the KMeans class from scikit-learn, how do I pick out what my k should be? That will be the number of clusters right? So 4 centroids

Yes: in K-Means, `k` is the number of clusters, and K-Means will learn `k` centroids.

So if you set:

```
python  
  
KMeans(n_clusters=4)
```

that means:

```
text  
  
4 clusters  
4 centroids
```

For this assignment, because the true obesity target has **7 classes**, a reasonable first choice is:

```
python  
  
n_clusters=7
```

That lets you compare:

```
text  
  
7 K-Means clusters vs 7 real obesity categories
```

Example:

```
python  
  
from sklearn.cluster import KMeans
```

If the TODO asks you to choose `k`, common approaches are:

Option 1: Use the known number of classes

Since `NObeyesdad` has 7 categories:

```
python
```

```
k = 7
```

This is simplest and easiest to justify.

Option 2: Elbow method

Try several `k` values and plot inertia:

```
python
```

```
inertias = []
```

```
for k in range(2, 11):
```

```
    kmeans = KMeans(n_clusters=k, random_state=RANDOM_STATE,
```

```
    kmeans.fit(obesity_scaled)
```

```
    inertias.append(kmeans.inertia_)
```

```
plt.plot(range(2, 11), inertias, marker="o")
```

```
plt.xlabel("Number of clusters (k)")
```

```
plt.ylabel("Inertia")
```

```
plt.title("Elbow Method for K-Means")
```

```
plt.show()
```

You look for the "elbow," where adding more clusters stops reducing inertia dramatically.

For your case, I'd start with:

```
python
```

```
n_clusters=7
```



because the assignment later asks you to compare clusters to real obesity

I did it and I don't know if its good, I also computed the silhouette score as well

Worked for 33s >

You're close, but the current K-Means loop is not doing what the TODO asks.

You have:

```
python
for k in range(2, 11):
    model = KMeans(n_clusters=7, random_state=RANDOM_STATE,
```

Even though the loop variable is `k`, you always use:

```
python
n_clusters=7
```

So your inertia list contains the same kind of model repeated. For the Elbow method, you need:

```
python
model = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_in
```

Conceptually:

```
python
inertias = []
silhouettes = []
```



Conceptually:

```
python

inertias = []
silhouettes = []

for k in range(2, 11):
    model = KMeans(n_clusters=k, random_state=RANDOM_STATE,
                    cluster_labels = model.fit_predict(obesity_scaled))

    inertias.append(model.inertia_)
    silhouettes.append(silhouette_score(obesity_scaled, clus
```

Then plot inertia:

```
python

plt.plot(range(2, 11), inertias, marker="o")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.show()
```

And optionally plot silhouette:

```
python

plt.plot(range(2, 11), silhouettes, marker="o")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Silhouette score")
plt.title("Silhouette Scores")
plt.show()
```

Think I am done, it is just left with the student reflection and final reflection questions.

But I need a top to bottom run down on the notebook to see if I have hit everything.

Worked for 56s >

Here's the top-to-bottom audit.

Overall

You have covered the main required tasks: taxi regression, obesity classification, K-Means clustering, PCA visualization, and crosstab comparison. Before submitting, I would do a full **Run All** from a clean kernel because most code cells currently show `execution_count=None`, so the saved notebook does not prove a clean top-to-bottom run yet.

Taxi Regression

Cell 6: Load/explore taxi data looks fine.

Cell 9: Cleaning and feature engineering mostly fine. You fixed the `trip_distance > 0` issue, so `tip_rate_proxy` should not create `inf`.

Potential issue: you define `categorical_columns`, but I do not see actual encoding for taxi categoricals. If `store_and_fwd_flag` is still text, `LinearRegression` should fail. Check:

```
python
X_train_ready.select_dtypes(include=["object", "string"]).co
```

If anything appears, encode it before model training.

Cell 12: Split and scaling logic is correct: fit on train only, transform val/test.

Cell 15: Regression has two important metric bugs:

`trip_distance > 0` issue, so `tip_rate_proxy` should not create `inf` .

Potential issue: you define `categorical_columns` , but I do not see actual encoding for taxi categoricals. If `store_and_fwd_flag` is still text, `LinearRegression` should fail. Check:

```
python
X_train_ready.select_dtypes(include=["object", "string"]).co
```

If anything appears, encode it before model training.

Cell 12: Split and scaling logic is correct: fit on train only, transform val/test.

Cell 15: Regression has two important metric bugs:

```
python
val_rmse = mean_squared_error(y_val, y_val_pred)
test_rmse = mean_squared_error(y_test, y_test_pred)
```

Those are MSE, not RMSE. They need `** 0.5` .

Also:

```
python
rf_train_rmse = mean_squared_error(y_train, y_train_pred) **
```

That uses the linear model prediction. It should use `rf_train_pred` .

Your plots and model comparison idea are good, but fix those metrics before writing the regression reflection.

Obesity Classification

Cell 19: Load/explore/class distribution looks good.

Cell 22: Encoding looks good now. You correctly use `obesity_clean = obesity.copy()` , map binary/ordinal  nder, one-hot encode `MTRANS` , create BMI, and encode the target.

python

```
obesity_scaled = scaler.fit_transform(obesity_before_scale)
```

That means K-Means labels and PCA coordinates are based on slightly different scaling approaches. Better conceptually: use the same `obesity_scaled` for K-Means and PCA. Do not overwrite it before PCA.

Cell 33: Crosstab is correct. It compares `cluster_labels` against the original `NObeyesdad`. Your interpretation is also right: some clusters map cleanly, especially `Obesity_Type_III`, while others are mixed.

Remaining Written Sections

You still need to fill these:

- Taxi data exploration reasoning
- Taxi preprocessing reasoning
- Taxi splitting reasoning
- Regression evaluation reasoning
- Obesity data exploration reasoning
- Obesity preprocessing reasoning
- Obesity splitting reasoning
- Classification evaluation reasoning
- Clustering reasoning
- Final reflection questions

Must Fix Before Submit

1. Fix taxi RMSE calculations.
2. Fix Random Forest train RMSE to use `rf_train_pred`.
3. Verify taxi categorical columns are encoded or absent from model input.
4. Make K-Means/PCA use the same `obesity_scaled`.
5. Restart kernel and Run All to confirm no hidden state is carrying the notebook.



6:01 PM

Trying to visualise something with seaborn, just a single column from a pandas dataframe, I wanted to copy it to a variable then use that. Is that necessary?

Thought for a couple of seconds >

No, it is **not** necessary.

With seaborn, you can use the column directly from the DataFrame.

For example, if your DataFrame is `df` and the column is `age`:

```
Python
import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(data=df, x="age")
plt.show()
```

That is usually the cleanest seaborn style.

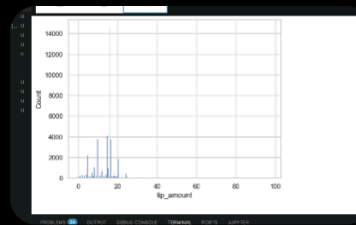
You could also do:

```
Python
sns.histplot(df["age"])
plt.show()
```

Copying it into a variable is only useful if you want readability or you plan to reuse it:

```
Python
age = df["age"]

sns.histplot(age)
plt.show()
```



Now it looks like this, is this normal?

Thought for 8s >

Yes, this can be normal, but it is telling you something important about the data.

From the plot, most `tip_amount` values seem to be clustered between roughly 0 and 25, but the x-axis goes all the way to about 100. That usually means there are a few large outliers stretching the axis, so the main part of the distribution looks squeezed.

Also, the very tall spike means many rows fall into that same bin/range.

Try checking the column first:

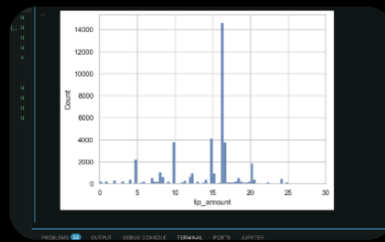
```
Python
df["tip_amount"].describe()
```

Also check the biggest values:

```
Python
df["tip_amount"].sort_values(ascending=False).head(20)
```

For a better histogram, limit the x-axis to the useful range:





Looks better now?

Thought for a few seconds >

Yes, this looks much better.

Now the main distribution is readable because you limited the x-axis to `0-30`, so the extreme outliers are no longer stretching the whole graph.

The tall spikes are still normal if many values repeat or cluster around certain amounts. For example, if `tip_amount` values are rounded, capped, or calculated in a way that produces repeated values, you will see sharp vertical bars like this.

One thing I would check now is whether `tip_amount` has lots of repeated exact values:

```
</> Python
df["tip_amount"].value_counts().head(20)
```



Run

Also, for this kind of column, using `binwidth` may make the histogram easier to interpret than just relying on automatic bins:

```
</> Python
sns.histplot(data=df, x="tip_amount", binwidth=1)
```



Run

Yes, this looks much better.

↑ Share

Now the main distribution is readable because you limited the x-axis to 0–30, so the extreme outliers are no longer stretching the whole graph.

The tall spikes are still normal if many values repeat or cluster around certain amounts. For example, if `tip_amount` values are rounded, capped, or calculated in a way that produces repeated values, you will see sharp vertical bars like this.

One thing I would check now is whether `tip_amount` has lots of repeated exact values:

```
</> Python
df["tip_amount"].value_counts().head(20)
```



▶ Run

Also, for this kind of column, using `binwidth` may make the histogram easier to interpret than just relying on automatic bins:

```
</> Python
sns.histplot(data=df, x="tip_amount", binwidth=1)

plt.xlim(0, 30)
plt.show()
```



▶ Run

Or smaller bins:

```
</> Python
sns.histplot(data=df, x="tip_amount", binwidth=0.5)

plt.xlim(0, 30)
plt.show()
```



▶ Run

So yes, the graph is more useful now. The next question is just whether those spikes are expected from the dataset or whether `tip_amount` has repeated/rounded values.

📄 🍌 🗨️ 🔗 ↺ ...